API Flask para regresión polinómica

Caso demostrativo: expedientes de terrazas en Madrid

IFCD107 – Especialista en Inteligencia Artificial
Proyecto práctico de API y aprendizaje automático

Octubre de 2026

Resumen

Este proyecto ofrece una API REST Flask con regresión polinómica, búsqueda de hiperparámetros mediante `RandomizedSearchCV` y Random Forest. La capa HTTP valida solicitudes; funciones auxiliares coordinan cada ejecución y las clases de `regression/` encapsulan los modelos. La colección Postman incluye ejemplos JSON y CSV.

1. Descripción de la solución

La aplicación ofrece dos familias de modelos: regresión polinómica sobre pares (x, y) y Random Forest para regresión tabular con variables numéricas y categóricas. La regresión polinómica también ofrece una ruta que compara configuraciones mediante validación cruzada. El ejemplo de dominio representa la gestión de expedientes de terrazas: x es un índice didáctico de complejidad y y son días de tramitación simulados. Los datos son sintéticos, no contienen expedientes reales ni reproducen estadísticas o plazos del Ayuntamiento de Madrid. Cada ejecución guarda la entrada, predicciones, gráfica, modelo y log.

Componente	Responsabilidad
`app.py`	Endpoints HTTP, validación, carpetas, persistencia y respuesta
`regression/polynomial_regressor.py`	Ajuste matemático, métricas, CSV de salida y gráfica.
`regression/random_forest_regressor.py`	Preprocesamiento mixto, entrenamiento RF e importancia de va
`scripts/generate_terrace_cases.py`	Generador determinista de las 1.000 filas sintéticas.
`ejemplos/datos_ejemplo_xy.csv`	Datos con columnas x, y para los endpoints polinómicos.
`postman/`	Colección importable con health, autenticación Bearer, solicitud
`runs/`	Artefactos generados, separados por referencia y ejecución.

2. Estructura de directorios

```
proyecto_regresion_polinomica/
  app.py
  api_functions.py
  regression/polynomial_regressor.py
  regression/random_forest_regressor.py
  ejemplos/datos_ejemplo_xy.csv
  ejemplos/datos_ejemplo_complejo.csv
  postman/Regresion_Polinomica.postman_collection.json
  documentacion/guia_proyecto.tex
  runs/<referencia>/
    graficas/<run_id>_ajuste.png
    csv_entrada/<run_id>.csv
    logs_salida/<run_id>.log
```

```
csv_salida/<run_id>_predicciones.csv
modelos/<run_id>_modelo.pkt
```

La referencia admite letras, números, punto, guion y guion bajo, con máximo de 64 caracteres. Se rechazan rutas o separadores para evitar escapar del directorio de ejecución. Cada petición genera un identificador único, llamado `run_id`; así se guardan varias ejecuciones sin sobrescribir los archivos de la misma referencia.

3. Instalación y arranque

Se recomienda Python 3.10 o superior. Desde la carpeta raíz del proyecto:

```
python -m venv .venv
```

Activar el entorno: en Windows PowerShell, `.venv Scripts Activate.ps1`; en macOS/Linux, `source .venv/bin/activate`. Después:

```
python -m pip install -r requirements.txt
```

La API carga automáticamente las variables del archivo local `.env` al arrancar. En esta copia se incluye un `.env` local con una contraseña aleatoria para el usuario `admin` y un secreto de firma. Los valores no se reproducen en esta guía. Para otro entorno, copia `.env.example` a `.env` y configura credenciales propias. El archivo `.env` está excluido de Git y del ZIP distribuible. Copia el valor `API_ADMIN_PASSWORD` a la variable `password` de Postman. Mantén `API_TOKEN_SECRET` estable entre procesos y reinicios.

La API queda disponible localmente en http://127.0.0.1:5000. http://127.0.0.1:5000/api/v1/health es público y devuelve el estado del servicio.

4. Flujo de ejecución

Para seguir el programa desde el principio, piense en cada petición como una cadena de pasos: Postman envía datos, Flask valida la petición, una función de `api_functions.py` ejecuta el modelo, y finalmente Flask devuelve JSON con el resultado y las rutas de los archivos creados.

1. **Arranque.** Al ejecutar `python app.py`, Flask crea la aplicación y registra sus rutas. La ruta pública `/api/v1/health` permite comprobar que el servidor responde.

2. **Autenticación.** El cliente envía usuario y contraseña a `/api/v1/auth/token`. Si coinciden con la configuración, la API devuelve un token firmado. En las peticiones de entrenamiento, el decorador `@require_token` comprueba el token antes de ejecutar la ruta.

3. **Lectura y validación.** Las rutas polinómicas aceptan puntos JSON o archivos CSV con encabezados `x,y`. Las rutas Random Forest reciben JSON para el conjunto de ejemplo o CSV multipart. Flask valida referencia, formato, tamaño y parámetros antes de iniciar el entrenamiento.

4. **Entrenamiento.** La ruta llama a una función en `api_functions.py`. Esa función divide los datos, construye el estimador o pipeline y calcula predicciones y métricas.

5. **Archivos y respuesta.** La función crea un `run_id`, guarda los artefactos bajo `runs/<referencia>/` y devuelve el identificador, métricas y rutas en una respuesta JSON.

4.1. Flujo de regresión polinómica

`POST /api/v1/regresion/polinomica` recibe JSON con `degree` y una lista `data`. La variante `/csv` recibe `reference`, `degree` y un archivo CSV. Ambas terminan en `run_model`: esta función separa entrenamiento y prueba, crea `PolynomialRegressor`, ajusta los coeficientes con NumPy y calcula predicciones y métricas para la partición de prueba. Después guarda el modelo, las predicciones, la gráfica y el log.

4.2. Flujo de RandomizedSearchCV

`POST /api/v1/regresion/polinomica/randomizedsearch` admite los dos formatos: JSON con `data`, o `multipart/form-data` con `file` (CSV con columnas `x,y`). En ambos casos también se pueden enviar `n_iter`, `cv` y `random_state`. El campo opcional `w` es una lista de pesos, uno por cada fila.

La ruta valida y convierte la entrada, y llama a `run_polynomial_search`. Esta función reserva primero el 20 % de las filas como prueba final. En el 80 % restante, `RandomizedSearchCV` muestrea combinaciones del diccionario `POLYNOMIAL_PARAM_DISTRIBUTIONS`; para cada combinación, valida el resultado en `cv` pliegues. El `Pipeline` conecta el adaptador `PolynomialSearchEstimator` con scikit-learn, mientras que el adaptador llama a `PolynomialRegressor`. Cuando termina la búsqueda, la mejor configuración se vuelve a ajustar con todos los datos de entrenamiento y se evalúa una sola vez con el conjunto de prueba reservado.

La respuesta de búsqueda incluye `best_parameters`, el RMSE promedio de validación cruzada, las métricas de prueba y las rutas de los archivos. El log también registra el diccionario de distribuciones y los parámetros del mejor estimador. `full` y `cov` controlan diagnósticos del ajuste; no son métricas de rendimiento, por lo que no cambian el criterio de selección (RMSE).

4.3. Flujo de Random Forest

Las rutas `/api/v1/random-forest/terrazas` y `/api/v1/random-forest/csv` llaman a `run_random_forest`. La clase `RandomForestRegressorModel` separa la columna objetivo de las predictoras, prepara las columnas numéricas y categóricas en un `ColumnTransformer`, y combina ese preprocesamiento con `RandomForestRegressor` en un `Pipeline`. El conjunto de prueba queda fuera del entrenamiento. La respuesta incluye métricas, las importancias principales y las rutas de los artefactos.

5. Prueba desde Postman

1. En Postman, importar `postman/Regresion_Polinomica.postman_collection.json`.
2. Mantener `baseUrl=http://127.0.0.1:5000` y usar la referencia `terrazas_madrid_demo`.
3. Configurar las variables de colección `username=admin` y `password` con la clave definida en `API_ADMIN_PASSWORD`.
4. Ejecutar **Health**; debe responder HTTP 200.
5. Ejecutar **Obtener token**. La colección guarda automáticamente el token Bearer recibido. Credenciales incorrectas producen HTTP 401.
6. Ejecutar **Ajustar desde JSON**. La colección ya incluye datos sintéticos y grado 2. Postman establece `Content-Type: application/json`.
7. Para los datos polinómicos CSV, ejecutar **Ajustar desde CSV** y seleccionar `ejemplos/datos_ejemplo_xy` Postman crea el formulario multipart; no fijar manualmente el encabezado boundary.
8. Ejecutar **Regresión polinómica - RandomizedSearchCV** para buscar parámetros con puntos JSON, o **RandomizedSearchCV - cargar CSV** para adjuntar `ejemplos/datos_ejemplo_xy.csv` Ambas solicitudes llaman a `/api/v1/regresion/polinomica/randomizedsearch`.

9. Ejecutar **Random Forest - CSV de terrazas incluido** para entrenar sin adjuntar fichero, o **Random Forest - cargar CSV** y seleccionar `ejemplos/datos_ejemplo_complejo.csv`. El objetivo por defecto es `y`; se excluye `expediente_id` y se usan como predictores el resto de columnas.

10. Leer `artifacts` en la respuesta JSON. Las rutas son relativas al directorio del proyecto y están agrupadas bajo `runs/<reference>/`.

5.1. Obtención del token

```
POST /api/v1/auth/token
Content-Type: application/json

{ "username": "admin", "password": "<clave configurada>" }
```

La respuesta incluye `access_token`, `token_type=Bearer` y `expires_in`. La duración predeterminada es una hora y se puede cambiar con `API_TOKEN_TTL_SECONDS`. Los endpoints de entrenamiento requieren el encabezado Bearer; sin token, con token inválido o expirado responden HTTP 401.

5.2. Solicitud JSON

```
POST /api/v1/regresion/polinomica
Authorization: Bearer <token>
Content-Type: application/json

{
  "reference": "expediente_demo_001",
  "degree": 2,
  "data": [
    {"x": -1, "y": 2.2}, {"x": 0, "y": 1.0},
    {"x": 1, "y": 1.8}, {"x": 2, "y": 6.2}
  ]
}
```

5.3. Solicitud CSV

```
POST /api/v1/regresion/polinomica/csv
Authorization: Bearer <token>
Body: form-data
  reference = terrazas_madrid_demo
  degree = 2
  file = ejemplos/datos_ejemplo_xy.csv (columnas x,y)
```

5.4. Solicitud de búsqueda polinómica

La ruta JSON y la ruta de archivo comparten el endpoint `/api/v1/regresion/polinomica/randomizedsearc`
La diferencia está en el formato de la petición:

```
# JSON: enviar puntos directamente en el cuerpo
POST /api/v1/regresion/polinomica/randomizedsearch
Authorization: Bearer <token>
Content-Type: application/json

{
  "reference": "busqueda_demo",
  "n_iter": 20,
```

```
  "cv": 5,
  "random_state": 42,
  "data": [
    {"x": 0, "y": 1}, {"x": 1, "y": 2}, {"x": 2, "y": 5},
    {"x": 3, "y": 10}, {"x": 4, "y": 17}, {"x": 5, "y": 26},
    {"x": 6, "y": 37}, {"x": 7, "y": 50}, {"x": 8, "y": 65},
    {"x": 9, "y": 82}
  ]
}

# CSV: en Postman elegir Body > form-data y adjuntar el fichero
reference = busqueda_csv_demo
n_iter = 20
cv = 5
random_state = 42
file = ejemplos/datos_ejemplo_xy.csv
```

El CSV debe tener los encabezados `x,y`. Si se usa el campo opcional `w`, se envía una lista JSON de pesos con tantos valores como filas tenga `data`; por ejemplo, `[1,1,0.5,...]`. Para JSON es un array normal; en multipart se escribe como texto con formato JSON.

5.5. Entrenamiento Random Forest

La ruta de demostración lee el CSV local `ejemplos/datos_ejemplo_complejo.csv` (1.000 filas sintéticas) y la ruta multipart admite otro CSV. Ambas requieren Bearer y guardan CSV de entrada, predicciones, gráfica, modelo `.pkt` y log en el mismo árbol `runs/<referencia>/`.

```
POST /api/v1/random-forest/terrazas
Authorization: Bearer <token>
Content-Type: application/json

{
  "reference": "terrazas_rf_demo",
  "target": "y",
  "n_estimators": 200,
  "max_depth": null,
  "test_size": 0.2,
  "random_state": 42
}
```

La ruta `/api/v1/random-forest/csv` acepta `multipart/form-data`: `reference`, `file` y, opcionalmente, `target`, `n_estimators`, `max_depth`, `test_size` y `random_state`. Se imputan valores ausentes por mediana en variables numéricas y por moda en categóricas; estas últimas se codifican con one-hot encoding. El identificador de expediente no se usa como predictor. La división aleatoria fija una semilla y separa train/test; las métricas se calculan únicamente en test. La respuesta incluye R^2, RMSE, MAE e importancia de las 15 variables transformadas más relevantes. Los datos del ejemplo son sintéticos y las métricas no describen el rendimiento real de la tramitación municipal.

6. Respuesta y archivos

La respuesta polinómica directa incluye grado, coeficientes y métricas. La búsqueda polinómica también devuelve los mejores parámetros encontrados, RMSE de validación cruzada y métricas sobre el conjunto de prueba; su archivo `<run_id>_randomized_search.pkt` contiene el mejor pipeline. La respuesta Random Forest incluye parámetros, columnas predictoras, métricas, las 15 importancias principales y rutas de artefactos. Todas las respuestas correctas incluyen `reference`, `run_id` y `artifacts`. El archivo polinómico directo `<run_id>_modelo.pkt` se puede recargar con `PolynomialRegressor.load(ruta)` para llamar a `predict(x)`; los pickles de búsqueda y Random

Forest guardan pipelines de scikit-learn. Los CSV de predicciones contienen observación, predicción y residuo; las gráficas muestran el ajuste. Los logs por ejecución registran parámetros, métricas e identidad auditada. Además, `runs/_auditoria/api_audit.jsonl` registra las peticiones HTTP, su ruta, estado, usuario, IP y huella truncada del token.

El formato `.pkt` de este ejemplo utiliza serialización Python pickle. Solo se deben cargar archivos generados localmente y de confianza: pickle puede ejecutar código al cargar un fichero manipulado. Para intercambio entre sistemas, conviene guardar los coeficientes y metadatos en JSON en vez de cargar pickles externos.

7. Validaciones y límites del ejemplo

La API limita el cuerpo HTTP a 5 MB, los archivos CSV a 4 MB y las entradas a 100.000 filas. Valida referencia, formato, finitud y parámetros; la búsqueda polinómica limita `n_iter` a 1–80, `cv` a 2–10 y exige que haya datos suficientes tras separar el conjunto de prueba. Responde HTTP 400 ante entrada no válida, 401 ante credenciales o token no válidos, 413 ante contenido demasiado grande y 422 cuando el ajuste no se puede completar. La ruta de salud es pública y las rutas de entrenamiento requieren Bearer.

8. Notas sobre interpretación

Random Forest no extrapola tendencias fuera del rango de entrenamiento de forma lineal y sus importancias no implican causalidad. La evaluación usa una partición aleatoria; no modela autocorrelación geográfica ni temporal.

La regresión polinómica es sensible al grado, al rango y escala de x, a valores atípicos y a extrapolación. R^2 y errores se calculan sobre los mismos datos usados para el ajuste; por sí solos no estiman generalización. Para uso predictivo real, separar entrenamiento y evaluación, comparar grados con validación y revisar residuos y supuestos del dominio.

Este caso sirve para demostrar el flujo técnico, no para estimar plazos reales de expedientes ni evaluar el desempeño de unidades municipales. Para ello harían falta datos autorizados, definición institucional del inicio/fin de tramitación, tratamiento de expedientes abiertos/censurados, análisis de estacionalidad y validación temporal fuera de muestra. Se puede regenerar el fichero con `python scripts/generate_terrace_cases.py`; la semilla fija mantiene la demo reproducible.

A. Código fuente completo de la API Flask

Este listado corresponde a `app.py`, que contiene la fábrica Flask y las rutas HTTP. La validación común y la ejecución se delegan en `api_functions.py`; los estimadores residen en módulos separados.

Listing 1: Código fuente completo de app.py

```
"""API Flask de ejemplo para ajuste de regresión polinómica."""
from __future__ import annotations

import hashlib
import hmac
import json
import logging
import os
import secrets
from functools import wraps
from datetime import datetime, timezone
from pathlib import Path
```

```
import numpy as np
from flask import Flask, g, jsonify, request
from itsdangerous import BadSignature, URLSafeTimedSerializer
from werkzeug.exceptions import RequestEntityTooLarge
from dotenv import load_dotenv

from api_functions import (
    parse_csv, parse_json_points, random_forest_options, run_model,
    run_random_forest, run_polynomial_search, validate_reference,
)


# Carga variables locales antes de inicializar el diccionario de usuarios y la
# configuración de caducidad. Las variables ya definidas en el entorno prevalecen.
load_dotenv(Path(__file__).resolve().parent / ".env", override=False)

# Todas las salidas quedan ancladas al proyecto, no al directorio desde el que
# se haya lanzado Flask. Esto evita que un cambio de working directory disperse
# artefactos de una misma ejecución.
OUTPUT_ROOT = Path(__file__).resolve().parent / "runs"
# Diccionario didáctico usuario -> contraseña. Las contraseñas se leen del entorno
# y no se escriben en el repositorio. Añade aquí más pares para otros usuarios.
USERS = {
    "admin": os.environ.get("API_ADMIN_PASSWORD", ""),
}
# Los tokens Bearer caducan tras una hora por defecto.
TOKEN_MAX_AGE_SECONDS = max(1, int(os.environ.get("API_TOKEN_TTL_SECONDS", "3600")))


def create_app(output_root: Path = OUTPUT_ROOT, users: dict | None = None, token_secret:
    str | None = None) -> Flask:
    app = Flask(__name__)
    output_root = Path(output_root)
    # API_TOKEN_SECRET debe ser estable y secreto en despliegues con varios
    # procesos. El secreto aleatorio permite arrancar el ejemplo localmente; en
    # ese caso los tokens dejan de ser válidos al reiniciar el proceso.
    token_secret = token_secret or os.environ.get("API_TOKEN_SECRET") or secrets.
        token_urlsafe(32)
    app.config["API_USERS"] = USERS if users is None else users
    app.config["API_TOKEN_MAX_AGE"] = TOKEN_MAX_AGE_SECONDS
    token_serializer = URLSafeTimedSerializer(token_secret, salt="regresion-polinomica-api
        -token")
    # Registro de auditoría central: incluye también login, health, errores 4xx
    # y rutas inexistentes, no solo las ejecuciones que crean un log por run.
    audit_dir = output_root / "_auditoria"
    audit_dir.mkdir(parents=True, exist_ok=True)
    audit_path = audit_dir / "api_audit.jsonl"
    audit_logger = logging.getLogger(
        "regression.api.audit." + hashlib.sha256(str(audit_path.resolve()).encode()).
            hexdigest()[:12]
    )
    audit_logger.setLevel(logging.INFO)
    audit_logger.propagate = False
    if not any(getattr(handler, "baseFilename", None) == str(audit_path.resolve())
               for handler in audit_logger.handlers):
        audit_handler = logging.FileHandler(audit_path, encoding="utf-8")
        audit_handler.setFormatter(logging.Formatter("%(message)s"))
        audit_logger.addHandler(audit_handler)
    # Flask rechaza cuerpos excesivos antes de parsearlos; el endpoint CSV aplica
    # además un máximo menor sobre el archivo concreto.
```

```
app.config["MAX_CONTENT_LENGTH"] = 5 * 1024 * 1024

def token_fingerprint(token: str) -> str:
    # Se registra solo una huella SHA-256, nunca el Bearer reutilizable.
    return hashlib.sha256(token.encode("utf-8")).hexdigest()[:16]

@app.before_request
def capture_request_context():
    authorization = request.headers.get("Authorization", "")
    scheme, separator, supplied_token = authorization.partition(" ")
    g.authenticated_user = None
    g.requested_username = None
    g.client_ip = request.remote_addr or "unknown"
    g.token_fingerprint = (
        token_fingerprint(supplied_token.strip())
        if separator and scheme.lower() == "bearer" and supplied_token.strip()
        else None
    )

@app.after_request
def write_audit_log(response):
    # JSONL mantiene una entrada estructurada y evita inyección de líneas en
    # los logs si usuario, ruta o cabeceras contienen caracteres especiales.
    record = {
        "timestamp_utc": datetime.now(timezone.utc).isoformat(),
        "method": request.method,
        "path": request.path,
        "status": response.status_code,
        "username": (getattr(g, "authenticated_user", None)
                     or getattr(g, "requested_username", None) or "anonymous"),
        "token_fingerprint": getattr(g, "token_fingerprint", None),
        "client_ip": getattr(g, "client_ip", None),
    }
    audit_logger.info(json.dumps(record, ensure_ascii=False, separators=(",", ":")))
    return response

@app.errorhandler(RequestEntityTooLarge)
def too_large(_error):
    return jsonify(error="El cuerpo supera el límite de 5 MB."), 413

@app.post("/api/v1/auth/token")
def issue_token():
    # El endpoint de autenticación es público; solo emite un token cuando
    # el par usuario/contraseña coincide con el diccionario configurado.
    body = request.get_json(silent=True)
    if not isinstance(body, dict):
        return jsonify(error="El cuerpo debe ser un objeto JSON."), 400
    username = body.get("username")
    password = body.get("password")
    if isinstance(username, str):
        g.requested_username = username
    if not isinstance(username, str) or not isinstance(password, str):
        return jsonify(error="username y password son obligatorios."), 400
    expected_password = app.config["API_USERS"].get(username)
    # compare_digest reduce diferencias de tiempo al comparar contraseñas.
    valid = (
        isinstance(expected_password, str)
        and bool(expected_password)
        and hmac.compare_digest(expected_password.encode("utf-8"), password.encode("utf
            -8"))
```

```
    )
    if not valid:
        return jsonify(error="Credenciales no válidas."), 401
    token = token_serializer.dumps({"sub": username})
    # La llamada de login también queda asociada a usuario y huella del
    # token recién emitido, sin registrar el valor secreto del token.
    g.authenticated_user = username
    g.token_fingerprint = token_fingerprint(token)
    return jsonify(
        access_token=token,
        token_type="Bearer",
        expires_in=app.config["API_TOKEN_MAX_AGE"],
    ), 200

def require_token(view):
    # Protege las rutas de entrenamiento sin duplicar la validación del token.
    @wraps(view)
    def wrapped(*args, **kwargs):
        authorization = request.headers.get("Authorization", "")
        scheme, separator, token = authorization.partition(" ")
        if not separator or scheme.lower() != "bearer" or not token.strip():
            response = jsonify(error="Se requiere Authorization: Bearer <token>.")
            response.status_code = 401
            response.headers["WWW-Authenticate"] = "Bearer"
            return response
        try:
            claims = token_serializer.loads(
                token.strip(), max_age=app.config["API_TOKEN_MAX_AGE"]
            )
        except BadSignature:
            response = jsonify(error="Token inválido o caducado.")
            response.status_code = 401
            response.headers["WWW-Authenticate"] = "Bearer"
            return response
        username = claims.get("sub") if isinstance(claims, dict) else None
        current_password = app.config["API_USERS"].get(username) if isinstance(username
            , str) else None
        if not isinstance(current_password, str) or not current_password:
            response = jsonify(error="Token inválido o caducado.")
            response.status_code = 401
            response.headers["WWW-Authenticate"] = "Bearer"
            return response
        g.authenticated_user = username
        return view(*args, **kwargs)
    return wrapped

@app.get("/api/v1/health")
def health():
    return jsonify(status="ok", service="regresion-polinomica-api", version="1.0.0")

@app.post("/api/v1/regresion/polinomica")
@require_token
def fit_json():
    # `silent=True` permite devolver un 400 uniforme para JSON mal formado,
    # en lugar de que el detalle interno del parser llegue al cliente.
    body = request.get_json(silent=True)
    if not isinstance(body, dict):
        return jsonify(error="El cuerpo debe ser un objeto JSON."), 400
    reference, error = validate_reference(body.get("reference"))
    if error:
```

```
        return jsonify(error=error), 400
    # El grado controla la complejidad y estabilidad del ajuste. Se limita
    # antes de construir el modelo para evitar modelos excesivamente flexibles.
    degree = body.get("degree", 2)
    if isinstance(degree, bool) or not isinstance(degree, int) or degree < 1 or degree
         > 10:
        return jsonify(error="degree debe ser un entero entre 1 y 10."), 400
    points = body.get("data")
    try:
        x, y = parse_json_points(points)
    except ValueError as exc:
        return jsonify(error=str(exc)), 400
    return run_model(app, output_root, reference, degree, x, y, "json")

@app.post("/api/v1/regresion/polinomica/csv")
@require_token
def fit_csv():
    # Multipart separa los parámetros de control del fichero binario y es el
    # formato que Postman usa para adjuntar CSV sin codificarlo en JSON.
    reference, error = validate_reference(request.form.get("reference"))
    if error:
        return jsonify(error=error), 400
    try:
        degree = int(request.form.get("degree", "2"))
    except (TypeError, ValueError):
        return jsonify(error="degree debe ser un entero entre 1 y 10."), 400
    if not 1 <= degree <= 10:
        return jsonify(error="degree debe ser un entero entre 1 y 10."), 400
    uploaded = request.files.get("file")
    if uploaded is None or not uploaded.filename:
        return jsonify(error="Falta el archivo CSV en el campo multipart 'file'."), 400
    # Se lee una sola vez y se controla el tamaño antes de decodificarlo; luego
    # se conserva el original recibido como rastro reproducible de la entrada.
    raw = uploaded.read()
    if len(raw) > 4 * 1024 * 1024:
        return jsonify(error="El CSV supera el límite de 4 MB."), 413
    try:
        text = raw.decode("utf-8-sig")
        x, y = parse_csv(text)
    except (UnicodeDecodeError, ValueError) as exc:
        return jsonify(error=f"CSV no válido: {exc}"), 400
    return run_model(app, output_root, reference, degree, x, y, "csv", raw_csv=raw)

@app.post("/api/v1/regresion/polinomica/randomizedsearch")
@require_token
def fit_polynomial_randomized_search():
    """Recibe datos JSON o CSV y busca la mejor configuración polinómica."""
    raw_csv = None
    # El mismo endpoint admite dos formatos: JSON lleva una lista de puntos;
    # multipart/form-data lleva los parámetros en form y el archivo en file.
    if request.is_json:
        source = request.get_json(silent=True)
        if not isinstance(source, dict):
            return jsonify(error="El cuerpo debe ser un objeto JSON."), 400
        try:
            x, y = parse_json_points(source.get("data"))
        except ValueError as exc:
            return jsonify(error=str(exc)), 400
    else:
        source = request.form
```

```
        uploaded = request.files.get("file")
        if uploaded is None or not uploaded.filename:
            return jsonify(error="Falta el archivo CSV en el campo multipart 'file'."),
                400
        raw_csv = uploaded.read()
        if len(raw_csv) > 4 * 1024 * 1024:
            return jsonify(error="El CSV supera el límite de 4 MB."), 413
        try:
            x, y = parse_csv(raw_csv.decode("utf-8-sig"))
        except (UnicodeDecodeError, ValueError) as exc:
            return jsonify(error=f"CSV no válido: {exc}"), 400

    reference, error = validate_reference(source.get("reference"))
    if error:
        return jsonify(error=error), 400

    def parse_integer(name, default, minimum, maximum):
        # Los números JSON llegan como int; los campos de un formulario llegan
        # como texto. Convertimos ambos formatos y después aplicamos el límite.
        raw = source.get(name, default)
        try:
            if isinstance(raw, bool) or (isinstance(raw, str) and not raw.strip()):
                raise ValueError
            parsed = int(raw)
            if isinstance(raw, float) and raw != parsed:
                raise ValueError
        except (TypeError, ValueError):
            raise ValueError(f"{name} debe ser un entero entre {minimum} y {maximum}.")
                from None
        if not minimum <= parsed <= maximum:
            raise ValueError(f"{name} debe ser un entero entre {minimum} y {maximum}.")
        return parsed

    try:
        n_iter = parse_integer("n_iter", 20, 1, 80)
        cv = parse_integer("cv", 5, 2, 10)
        random_state = parse_integer("random_state", 42, 0, 2**32 - 1)
    except ValueError as exc:
        return jsonify(error=str(exc)), 400
    if cv >= int(len(x) * 0.8):
        return jsonify(error="cv debe ser menor que el número de observaciones de
            entrenamiento (80% de data)."), 400
    # Los pesos siguen el mismo orden que las filas de entrada. El ejecutor
    # recorta este vector junto con los datos en cada partición train/test.
    weights = source.get("w")
    if weights is not None:
        if isinstance(weights, str):
            try:
                weights = json.loads(weights)
            except json.JSONDecodeError:
                return jsonify(error="w debe ser una lista JSON de pesos numéricos."),
                    400
        try:
            weights = np.asarray(weights, dtype=float)
        except (TypeError, ValueError):
            return jsonify(error="w debe ser una lista numérica con un peso por
                observación."), 400
        if weights.ndim != 1 or len(weights) != len(x) or not np.isfinite(weights).all
            () or np.any(weights < 0):
            return jsonify(error="w debe contener pesos finitos no negativos, uno por
```

```
                observación."), 400
    options = {"n_iter": n_iter, "cv": cv, "random_state": random_state}
    return run_polynomial_search(output_root, reference, x, y, options, weights=
        weights, raw_csv=raw_csv)


@app.post("/api/v1/random-forest/terrazas")
@require_token
def fit_random_forest_terraces():
    # Emplea el CSV versionado en ejemplos para reproducir la práctica sin subir
        fichero.
    body = request.get_json(silent=True)
    if body is None:
        body = {}
    if not isinstance(body, dict):
        return jsonify(error="El cuerpo debe ser un objeto JSON."), 400
    reference, error = validate_reference(body.get("reference"))
    if error:
        return jsonify(error=error), 400
    try:
        options = random_forest_options(body)
    except ValueError as exc:
        return jsonify(error=str(exc)), 400
    sample_path = Path(__file__).resolve().parent / "ejemplos" / "
        datos_ejemplo_complejo.csv"
    if not sample_path.is_file():
        return jsonify(error="No está disponible el CSV de ejemplo complejo."), 500
    return run_random_forest(app, output_root, reference, sample_path.read_bytes(),
        options,
                             "datos_ejemplo_complejo.csv")

@app.post("/api/v1/random-forest/csv")
@require_token
def fit_random_forest_csv():
    # Ruta multipart para entrenar con un CSV del cliente usando el mismo pipeline.
    reference, error = validate_reference(request.form.get("reference"))
    if error:
        return jsonify(error=error), 400
    try:
        options = random_forest_options(request.form)
    except ValueError as exc:
        return jsonify(error=str(exc)), 400
    uploaded = request.files.get("file")
    if uploaded is None or not uploaded.filename:
        return jsonify(error="Falta el archivo CSV en el campo multipart 'file'."), 400
    raw = uploaded.read()
    if len(raw) > 4 * 1024 * 1024:
        return jsonify(error="El CSV supera el límite de 4 MB."), 413
    return run_random_forest(app, output_root, reference, raw, options, uploaded.
        filename)

return app
```

```
app = create_app()

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, debug=False)
```

B. Funciones auxiliares de la API

`api_functions.py` centraliza las validaciones, el parseo tabular y la persistencia de resultados compartida por los endpoints.

Listing 2: Código de api_functions.py

```
"""Funciones de validación y ejecución que comparten los endpoints Flask."""
from __future__ import annotations

import csv
import io
import logging
import re
from datetime import datetime, timezone
from pathlib import Path
from uuid import uuid4
from sklearn.model_selection import train_test_split
import json
import numpy as np
from flask import g, jsonify

from regression.polynomial_regressor import PolynomialRegressor,build_polynomial_search
from regression.random_forest_regressor import RandomForestRegressorModel

# Límites del procesamiento aplicados tanto a endpoints JSON como a CSV.
MAX_POINTS = 100_000
REFERENCE_RE = re.compile(r"^[A-Za-z0-9][A-Za-z0-9._-]{0,63}$")

def validate_reference(value):
    # La validación se aplica antes de formar cualquier ruta. No se normaliza una
    # referencia inválida porque eso podría hacer colisionar expedientes distintos.
    if not isinstance(value, str) or not REFERENCE_RE.fullmatch(value):
        return None, "reference es obligatorio y debe tener 1-64 caracteres: letras, nú
            meros, punto, guion o guion bajo."
    return value, None

def parse_json_points(points):
    # Limitar número de filas antes de convertirlas a arrays evita asignaciones
    # grandes no necesarias; después se comprueba tipo y finitud en validate_xy.
    if not isinstance(points, list) or not 2 <= len(points) <= MAX_POINTS:
        raise ValueError(f"data debe contener entre 2 y {MAX_POINTS} pares x/y.")
    try:
        x = np.asarray([row["x"] for row in points], dtype=float)
        y = np.asarray([row["y"] for row in points], dtype=float)
    except (TypeError, ValueError, KeyError, IndexError):
```

```
        raise ValueError("Cada elemento de data debe tener valores numéricos x e y.") from
            None
    return validate_xy(x, y)

def parse_csv(text):
    # DictReader permite conservar columnas contextuales del caso de terrazas,
    # aunque el algoritmo solo use x e y. Las filas extra no se descartan del
    # CSV archivado; solo se extraen aquí las dos variables del modelo.
    reader = csv.DictReader(io.StringIO(text))
    if not reader.fieldnames or not {"x", "y"}.issubset(set(reader.fieldnames)):
        raise ValueError("El encabezado debe contener las columnas x,y.")
    x_values, y_values = [], []
    for line_number, row in enumerate(reader, start=2):
        try:
            x_values.append(float(row["x"]))
            y_values.append(float(row["y"]))
        except (TypeError, ValueError):
            raise ValueError(f"Valores no numéricos en la línea {line_number}.") from None
        if len(x_values) > MAX_POINTS:
            raise ValueError(f"El CSV supera {MAX_POINTS} filas de datos.")
    return validate_xy(np.asarray(x_values), np.asarray(y_values))

def validate_xy(x, y):
    # NaN e infinito contaminan coeficientes y métricas sin producir siempre una
    # excepción clara; se rechazan en el borde HTTP antes de invocar NumPy.
    if x.ndim != 1 or y.ndim != 1 or len(x) != len(y) or len(x) < 2:
        raise ValueError("Se requieren al menos dos pares x/y del mismo tamaño.")
    if not np.isfinite(x).all() or not np.isfinite(y).all():
        raise ValueError("x e y solo pueden contener números finitos.")
    return x, y

def random_forest_options(source):
    """Valida opciones para el formato JSON o multipart de Postman."""
    def parse_int(key, default):
        raw = source.get(key, default)
        try:
            parsed = int(raw)
            if isinstance(raw, float) and raw != parsed:
                raise ValueError
            return parsed
        except (TypeError, ValueError):
            raise ValueError(f"{key} debe ser un entero válido.") from None
    n_estimators = parse_int("n_estimators", 200)
    random_state = parse_int("random_state", 42)
    raw_depth = source.get("max_depth", None)
    max_depth = None if raw_depth in (None, "", "null") else parse_int("max_depth", None)
    try:
        test_size = float(source.get("test_size", 0.2))
    except (TypeError, ValueError):
        raise ValueError("test_size debe ser un número válido.") from None
    target = source.get("target", "y")
    if not 10 <= n_estimators <= 1000:
        raise ValueError("n_estimators debe estar entre 10 y 1000.")
    if max_depth is not None and not 1 <= max_depth <= 100:
        raise ValueError("max_depth debe ser null o estar entre 1 y 100.")
    if not 0.1 <= test_size <= 0.4:
        raise ValueError("test_size debe estar entre 0.1 y 0.4.")
    if random_state < 0:
        raise ValueError("random_state debe ser un entero no negativo.")
    if not isinstance(target, str) or not target.strip():
```

```
        raise ValueError("target debe indicar el nombre de una columna.")
    return {"n_estimators": n_estimators, "max_depth": max_depth, "test_size": test_size,
            "random_state": random_state, "target": target}

def run_random_forest(app, output_root, reference, raw_csv, options, source_name):
    """Ejecuta el pipeline y guarda artefactos separados por referencia/run_id."""
    from io import BytesIO
    import pandas as pd

    run_id = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ") + "_" + uuid4().hex
        [:8]
    base = Path(output_root) / reference
    folders = {name: base / name for name in ("graficas", "csv_entrada", "logs_salida", "
        csv_salida", "modelos")}
    for folder in folders.values():
        folder.mkdir(parents=True, exist_ok=True)
    logger = logging.getLogger(f"random_forest.{reference}.{run_id}")
    logger.setLevel(logging.INFO)
    handler = logging.FileHandler(folders["logs_salida"] / f"{run_id}.log", encoding="utf
        -8")
    handler.setFormatter(logging.Formatter("%(asctime)s %(levelname)s %(message)s"))
    logger.addHandler(handler)
    try:
        # Vincula cada entrenamiento a la identidad auditada por la capa Flask.
        logger.info("Petición algoritmo=random_forest username=%s client_ip=%s
            token_fingerprint=%s source=%s",
                getattr(g, "authenticated_user", None) or "anonymous",
                getattr(g, "client_ip", None) or "unknown",
                getattr(g, "token_fingerprint", None) or "none", source_name)
        input_path = folders["csv_entrada"] / f"{run_id}.csv"
        input_path.write_bytes(raw_csv)
        try:
            frame = pd.read_csv(BytesIO(raw_csv), encoding="utf-8-sig")
        except (UnicodeDecodeError, pd.errors.ParserError, pd.errors.EmptyDataError) as
            exc:
            raise ValueError(f"CSV no válido: {exc}") from None
        if len(frame) > MAX_POINTS:
            raise ValueError(f"El CSV supera {MAX_POINTS} filas de datos.")
        if frame.columns.duplicated().any():
            raise ValueError("El CSV no puede contener nombres de columna duplicados.")
        model = RandomForestRegressorModel(n_estimators=options["n_estimators"],
                                           max_depth=options["max_depth"],
                                           test_size=options["test_size"],
                                           random_state=options["random_state"])
        result = model.fit(frame, target=options["target"])
        csv_path = folders["csv_salida"] / f"{run_id}_predicciones.csv"
        graph_path = folders["graficas"] / f"{run_id}_random_forest.png"
        model_path = folders["modelos"] / f"{run_id}_random_forest.pkt"
        model.save(model_path, result)
        model.write_predictions(csv_path, result)
        model.write_plot(graph_path, result)
        metadata = {
            "reference": reference, "run_id": run_id,
            "algorithm": "RandomForestRegressor", "target": result.target,
            "feature_columns": result.feature_columns,
            "parameters": {key: options[key] for key in ("n_estimators", "max_depth", "
                test_size", "random_state")},
            "n_rows_input": int(len(frame)), "n_rows_train": int(len(frame) - len(result.
                predictions)),
            "n_rows_test": int(len(result.predictions)), "metrics": result.metrics,
```

```
            "feature_importances_top_15": result.feature_importances[:15],
            "artifacts": {
                "input_csv": str(input_path.relative_to(output_root)),
                "output_csv": str(csv_path.relative_to(output_root)),
                "graph_png": str(graph_path.relative_to(output_root)),
                "trained_model_pkt": str(model_path.relative_to(output_root)),
                "log": str((folders["logs_salida"] / f"{run_id}.log").relative_to(
                    output_root)),
            },
        }
        logger.info("Entrenamiento Random Forest completado: n=%d metrics=%s", len(frame),
             result.metrics)
        return jsonify(metadata), 200
    except (ValueError, UnicodeDecodeError) as exc:
        logger.exception("No se pudo entrenar el Random Forest: %s", exc)
        return jsonify(error=f"No se pudo entrenar el modelo: {exc}", reference=reference,
             run_id=run_id), 422
    except Exception as exc:
        logger.exception("Error interno entrenando Random Forest: %s", exc)
        return jsonify(error="Error interno durante el entrenamiento.", reference=
            reference, run_id=run_id), 500
    finally:
        handler.close()
        logger.removeHandler(handler)

def run_model(app, output_root, reference, degree, x, y, input_kind, raw_csv=None):
    # El sufijo aleatorio evita sobrescrituras si llegan dos peticiones para la
    # misma referencia dentro del mismo segundo. UTC facilita correlación de logs.
    run_id = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ") + "_" + uuid4().hex
        [:8]
    base = Path(output_root) / reference
    # Un directorio raíz por referencia y carpetas por tipo de artefacto hacen
    # posible auditar/reprocesar cada caso sin mezclar entrada, modelo y salida.
    folders = {name: base / name for name in ("graficas", "csv_entrada", "logs_salida", "
        csv_salida", "modelos")}
    for folder in folders.values():
        folder.mkdir(parents=True, exist_ok=True)
    logger = logging.getLogger(f"regression.{reference}.{run_id}")
    logger.setLevel(logging.INFO)
    handler = logging.FileHandler(folders["logs_salida"] / f"{run_id}.log", encoding="utf
        -8")
    handler.setFormatter(logging.Formatter("%(asctime)s %(levelname)s %(message)s"))
    logger.addHandler(handler)
    try:
        # Cada log de ejecución enlaza el trabajo con usuario, IP y huella de
        # token de la petición autenticada, sin almacenar el Bearer en claro.
        logger.info(
            "Petición username=%s client_ip=%s token_fingerprint=%s",
            getattr(g, "authenticated_user", None) or "anonymous",
            getattr(g, "client_ip", None) or "unknown",
            getattr(g, "token_fingerprint", None) or "none",
        )
        input_path = folders["csv_entrada"] / f"{run_id}.csv"
        if raw_csv is None:
            # Para JSON también se guarda una entrada tabular mínima; así cada run
            # tiene una fuente de datos recuperable independientemente del canal.
            with input_path.open("w", newline="", encoding="utf-8") as stream:
                writer = csv.writer(stream)
                writer.writerow(["x", "y"])
                writer.writerows(zip(x.tolist(), y.tolist()))
```

```
        else:
            # Guardar bytes originales preserva columnas extra, orden, formato y
            # encabezados exactamente como los envió el cliente.
            input_path.write_bytes(raw_csv)


        X_train, X_test, y_train, y_test = train_test_split(
            x, y, test_size=0.20, random_state=42
        )
        model = PolynomialRegressor(X_train, X_test, y_train, y_test,degree=degree)
        result = model.fit()
        csv_path = folders["csv_salida"] / f"{run_id}_predicciones.csv"
        graph_path = folders["graficas"] / f"{run_id}_ajuste.png"
        model_path = folders["modelos"] / f"{run_id}_modelo.pkt"
        # Persistir el modelo inmediatamente tras entrenar permite volver a usarlo
        # aunque la API se reinicie; el endpoint devuelve la ruta para trazabilidad.
        model.save(model_path)
        model.write_predictions(csv_path, result)
        model.write_plot(graph_path, X_test, y_test, result)
        metadata = {
            "reference": reference,
            "run_id": run_id,
            "degree": degree,
            "n_observations": int(len(X_test)),
            "coefficients_descending": result.coefficients.tolist(),
            "metrics": result.metrics,
            "artifacts": {
                "input_csv": str(input_path.relative_to(output_root)),
                "output_csv": str(csv_path.relative_to(output_root)),
                "graph_png": str(graph_path.relative_to(output_root)),
                "trained_model_pkt": str(model_path.relative_to(output_root)),
                "log": str((folders["logs_salida"] / f"{run_id}.log").relative_to(
                    output_root)),
            },
        }
        logger.info("Ajuste completado input=%s degree=%d n=%d metrics=%s", input_kind,
            degree, len(x), result.metrics)
        return jsonify(metadata), 200
    except (ValueError, np.linalg.LinAlgError) as exc:
        # Errores de datos/ajuste se registran con el run_id y reciben 422; el log
        # conserva traceback para diagnóstico sin mezclarlo con la respuesta normal.
        logger.exception("No se pudo ajustar el modelo: %s", exc)
        return jsonify(error=f"No se pudo ajustar el modelo: {exc}", reference=reference,
            run_id=run_id), 422
    except Exception as exc: # No devolver rutas, stack traces ni secretos al cliente.
        logger.exception("Error interno en el ajuste: %s", exc)
        return jsonify(error="Error interno durante el ajuste.", reference=reference,
            run_id=run_id), 500
    finally:
        # Cerrar el handler por solicitud evita descriptores abiertos durante un
        # servicio prolongado y permite leer/archivar el log al terminar la petición.
        handler.close()
        logger.removeHandler(handler)


def run_polynomial_search(output_root, reference, x, y, options, weights=None, raw_csv=
    None):
    """Separa los datos, busca el modelo y guarda sus resultados para la API."""
    from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
    from sklearn.model_selection import train_test_split
```

```
import pickle
import matplotlib.pyplot as plt

run_id = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ") + "_" + uuid4().hex
    [:8]
base = Path(output_root) / reference
folders = {name: base / name for name in ("graficas", "csv_entrada", "logs_salida", "
    csv_salida", "modelos")}
for folder in folders.values():
    folder.mkdir(parents=True, exist_ok=True)
logger = logging.getLogger(f"polynomial_search.{reference}.{run_id}")
logger.setLevel(logging.INFO)
handler = logging.FileHandler(folders["logs_salida"] / f"{run_id}.log", encoding="utf
    -8")
handler.setFormatter(logging.Formatter("%(asctime)s %(levelname)s %(message)s"))
logger.addHandler(handler)
try:
    source_kind = "csv" if raw_csv is not None else "json"
    logger.info("RandomizedSearchCV user=%s client_ip=%s source=%s",
                getattr(g, "authenticated_user", None) or "anonymous",
                getattr(g, "client_ip", None) or "unknown", source_kind)
    input_path = folders["csv_entrada"] / f"{run_id}.csv"
    if raw_csv is not None:
        input_path.write_bytes(raw_csv)
    else:
        with input_path.open("w", newline="", encoding="utf-8") as stream:
            writer = csv.writer(stream)
            writer.writerow(["x", "y"])
            writer.writerows(zip(x.tolist(), y.tolist()))

    # Apartamos un 20% para medir el rendimiento final. RandomizedSearchCV
    # solo ve el 80% restante, así la prueba no influye en la selección.
    indices = np.arange(len(x))
    train_idx, test_idx = train_test_split(
        indices, test_size=0.2, random_state=options["random_state"]
    )
    # El pipeline prueba combinaciones usando validación cruzada dentro del
    # conjunto de entrenamiento y vuelve a ajustar la ganadora (refit=True).
    search = build_polynomial_search(
        n_iter=options["n_iter"], cv=options["cv"], random_state=options["random_state"
            ]
    )
    X_train = x[train_idx].reshape(-1, 1)
    X_test = x[test_idx].reshape(-1, 1)
    fit_kwargs = {}
    if weights is not None:
        # Pipeline necesita el prefijo del paso para enviar los pesos al modelo.
        fit_kwargs["polynomial__sample_weight"] = weights[train_idx]
    search.fit(X_train, y[train_idx], **fit_kwargs)
    # La evaluación se calcula con datos que no participaron en la búsqueda.
    predicted = search.best_estimator_.predict(X_test)
    observed = y[test_idx]
    residuals = observed - predicted
    metrics = {
        "r2": float(r2_score(observed, predicted)),
        "rmse": float(np.sqrt(mean_squared_error(observed, predicted))),
        "mae": float(mean_absolute_error(observed, predicted)),
    }
    csv_path = folders["csv_salida"] / f"{run_id}_predicciones.csv"
    np.savetxt(csv_path, np.column_stack((x[test_idx], observed, predicted, residuals)
```

```
        ),
              delimiter=",", header="x,y_observado,y_predicho,residuo", comments="",
                  fmt="%.10g")
    graph_path = folders["graficas"] / f"{run_id}_randomized_search.png"
    order = np.argsort(x[test_idx])
    fig, ax = plt.subplots(figsize=(9, 5.5))
    ax.scatter(x[test_idx], observed, label="Observaciones", color="#1f77b4", alpha
        =0.85)
    ax.plot(x[test_idx][order], predicted[order], label="Mejor ajuste", color="#d62728
        ")
    ax.set(title="Regresión polinómica optimizada", xlabel="x", ylabel="y")
    ax.grid(True, alpha=0.25)
    ax.legend()
    fig.tight_layout()
    fig.savefig(graph_path, dpi=160)
    plt.close(fig)
    model_path = folders["modelos"] / f"{run_id}_randomized_search.pkt"
    with model_path.open("wb") as stream:
        pickle.dump(search.best_estimator_, stream, protocol=pickle.HIGHEST_PROTOCOL)
    metadata = {
        "reference": reference,
        "run_id": run_id,
        "algorithm": "PolynomialRegressionRandomizedSearchCV",
        "best_parameters": search.best_params_,
        "best_cv_rmse": float(np.sqrt(-search.best_score_)),
        "metrics": metrics,
        "n_rows_input": int(len(x)),
        "n_rows_train": int(len(train_idx)),
        "n_rows_test": int(len(test_idx)),
        "artifacts": {
            "input_csv": str(input_path.relative_to(output_root)),
            "output_csv": str(csv_path.relative_to(output_root)),
            "graph_png": str(graph_path.relative_to(output_root)),
            "trained_model_pkt": str(model_path.relative_to(output_root)),
            "log": str((folders["logs_salida"] / f"{run_id}.log").relative_to(
                output_root)),
        },
    }
    best_estimator_params = search.best_estimator_.named_steps["polynomial"].
        get_params(deep=False)
    logger.info(
        "RandomizedSearchCV completado param_distributions=%s best_estimator_params=%s
             best=%s metrics=%s",
        json.dumps(search.param_distributions, sort_keys=True, default=str),
        json.dumps(best_estimator_params, sort_keys=True, default=str),
        search.best_params_, metrics,
    )
    return jsonify(metadata), 200
except (ValueError, np.linalg.LinAlgError) as exc:
    logger.exception("No se pudo entrenar RandomizedSearchCV: %s", exc)
    return jsonify(error=f"No se pudo entrenar el modelo: {exc}", reference=reference,
         run_id=run_id), 422
except Exception:
    logger.exception("Error interno en RandomizedSearchCV")
    return jsonify(error="Error interno durante la búsqueda del modelo.", reference=
        reference, run_id=run_id), 500
finally:
    handler.close()
    logger.removeHandler(handler)
```

C. Clase de regresión polinómica

Este listado muestra el estimador clásico y su adaptador para el pipeline de búsqueda. Los comentarios del código explican el papel de cada paso.

Listing 3: Código de regression/polynomial_regressor.py

```
"""Clase de regresión polinómica y generación de artefactos."""
from dataclasses import dataclass
import pickle
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
from sklearn.base import BaseEstimator, RegressorMixin
from sklearn.model_selection import RandomizedSearchCV
from sklearn.pipeline import Pipeline




@dataclass
class FitResult:
    coefficients: np.ndarray
    x: np.ndarray
    predicted: np.ndarray
    residuals: np.ndarray
    metrics: dict
    rank: int | None = None
    singular_values: np.ndarray | None = None
    rcond: float | None = None
    covariance: np.ndarray | None = None
    fit_residuals: np.ndarray | None = None


class PolynomialRegressor:
    """Ajusta un polinomio por mínimos cuadrados usando numpy."""

    def __init__(self, X_train, X_test, y_train, y_test,degree: int = 2):
        if isinstance(degree, bool) or not isinstance(degree, int) or not 1 <= degree <=
            10:
            raise ValueError("degree debe estar entre 1 y 10.")
        self.degree = degree
        self.coefficients_ = None
        self.X_train = X_train
        self.X_test = X_test
        self.y_train = y_train
        self.y_test = y_test

    def fit(self, rcond=None, full: bool = False, w=None, cov=False) -> FitResult:
        """Ajusta el polinomio; los argumentos siguen las opciones de ``numpy.polyfit``.

        ``full=True`` incluye rango, valores singulares y residuos del ajuste en
        FitResult. ``cov`` puede ser True (covarianza escalada) o ``'unscaled'``.
        """
        if not isinstance(full, (bool, np.bool_)):
            raise TypeError("full debe ser booleano.")
```

```
if not (isinstance(cov, (bool, np.bool_)) or (isinstance(cov, str) and cov == "
    unscaled")):
    raise ValueError("cov debe ser False, True o 'unscaled'.")
cov = "unscaled" if isinstance(cov, str) else bool(cov)
# Convertimos a arrays numéricos una sola vez. x/y de train aprenden los
# coeficientes; x/y de test se reservan para medir el ajuste.
x = np.asarray(self.X_train, dtype=float)
y = np.asarray(self.y_train, dtype=float)
x_t = np.asarray(self.X_test, dtype=float)
y_t = np.asarray(self.y_test, dtype=float)
if x.ndim != 1 or y.ndim != 1 or len(x) != len(y):
    raise ValueError("x e y deben ser vectores de igual longitud.")
# Para grado d se requieren al menos d+1 observaciones y d+1 valores x
# distintos; de lo contrario el problema está subdeterminado.
if len(x) <= self.degree:
    raise ValueError("Se necesitan más observaciones que el grado del polinomio.")
if not np.isfinite(x).all() or not np.isfinite(y).all():
    raise ValueError("Los datos deben ser finitos.")
if np.unique(x).size <= self.degree:
    raise ValueError("Se necesitan más valores x distintos que el grado del
        polinomio.")
weights = None if w is None else np.asarray(w, dtype=float)
if weights is not None and (weights.ndim != 1 or len(weights) != len(x)):
    raise ValueError("w debe ser un vector con un peso por observación de
        entrenamiento.")
if weights is not None and (not np.isfinite(weights).all() or np.any(weights < 0))
    :
    raise ValueError("w debe contener pesos finitos no negativos.")
if rcond is not None:
    if isinstance(rcond, bool) or not np.isscalar(rcond):
        raise ValueError("rcond debe ser un número finito no negativo o None.")
    try:
        rcond = float(rcond)
    except (TypeError, ValueError):
        raise ValueError("rcond debe ser un número finito no negativo o None.")
            from None
    if not np.isfinite(rcond) or rcond < 0:
        raise ValueError("rcond debe ser un número finito no negativo o None.")
# polyfit calcula los coeficientes. w pondera los residuos; full pide
# diagnósticos SVD y cov pide la incertidumbre de los coeficientes.
fitted = np.polyfit(x, y, self.degree, rcond=rcond, full=bool(full), w=weights,
    cov=cov)
if full:
    coefficients, fit_residuals, rank, singular_values, used_rcond = fitted
    covariance = None
elif cov is not False:
    coefficients, covariance = fitted
    fit_residuals = np.array([], dtype=float)
    rank = None
    singular_values = None
    used_rcond = rcond
else:
    coefficients = fitted
    covariance = None
    fit_residuals = np.array([], dtype=float)
    rank = None
    singular_values = None
    used_rcond = rcond
self.coefficients_ = coefficients.copy()
# Evaluamos con X_test, que no se usó para calcular los coeficientes.
```

```
        predicted = np.polyval(coefficients, x_t)
        # El residuo es el valor real menos la predicción para cada fila.
        residuals = y_t - predicted
        # R no está definido si y es constante. En el caso perfecto se informa 1;
        # si no lo es, se informa 0 como valor neutro y se conserva RMSE/MAE.
        ss_res = float(np.sum(residuals ** 2))
        ss_tot = float(np.sum((y_t - np.mean(y_t)) ** 2))
        r2 = 1.0 if ss_tot == 0 and ss_res == 0 else (0.0 if ss_tot == 0 else 1 - ss_res /
             ss_tot)
        metrics = {
            "r2": float(r2),
            "rmse": float(np.sqrt(np.mean(residuals ** 2))),
            "mae": float(np.mean(np.abs(residuals))),
        }
        return FitResult(np.asarray(coefficients), x_t.copy(), np.asarray(predicted),
            residuals, metrics,
                        rank, np.asarray(singular_values) if singular_values is not None
                            else None,
                        float(used_rcond) if used_rcond is not None else None,
                        np.asarray(covariance) if covariance is not None else None,
                        np.asarray(fit_residuals) if full else None)

    def predict(self, x):
        """Predice con un modelo ya entrenado; x puede ser escalar o vector."""
        if self.coefficients_ is None:
            raise ValueError("El modelo aún no está entrenado.")
        values = np.asarray(x, dtype=float)
        if not np.isfinite(values).all():
            raise ValueError("x debe contener valores finitos.")
        # Los coeficientes se guardan en orden descendente, que es el convenio
        # esperado por polyval: a_n*x^n + ... + a_1*x + a_0.
        return np.polyval(self.coefficients_, values)

    def save(self, path: Path):
        """Serializa el modelo entrenado como pickle con extensión .pkt.

        Cargar archivos pickle únicamente si proceden de una fuente confiable.
        """
        if self.coefficients_ is None:
            raise ValueError("No se puede guardar un modelo sin entrenar.")
        path = Path(path)
        # Pickle conserva el objeto Python entrenado y sus coeficientes. El archivo
        # solo debe cargarse desde un origen confiable; pickle no es un formato
        # seguro para modelos recibidos de usuarios o servicios externos.
        with path.open("wb") as stream:
            pickle.dump(self, stream, protocol=pickle.HIGHEST_PROTOCOL)

    @classmethod
    def load(cls, path: Path):
        """Carga un .pkt creado por esta clase. No cargar archivos no confiables."""
        # Un pickle puede ejecutar código durante unpickle: validar el tipo después
        # no neutraliza un archivo malicioso, por eso el origen debe ser confiable.
        with Path(path).open("rb") as stream:
            model = pickle.load(stream)
        if not isinstance(model, cls) or model.coefficients_ is None:
            raise ValueError("El archivo no contiene un modelo PolynomialRegressor
                entrenado.")
        return model

    @staticmethod
```

```
    def write_predictions(path: Path, result: FitResult):
        path = Path(path)
        # La salida conserva observación, predicción y residuo fila a fila para que
        # el usuario pueda revisar errores y no solo el resumen agregado del modelo.
        values = np.column_stack((result.x, result.predicted + result.residuals, result.
            predicted, result.residuals))
        np.savetxt(path, values, delimiter=",", header="x,y_observado,y_predicho,residuo",
             comments="", fmt="%.10g")

    def write_plot(self, path: Path, x, y, result: FitResult):
        path = Path(path)
        # Ordenar solo para dibujar la curva evita segmentos zigzagueantes cuando
        # las observaciones llegan desordenadas; el ajuste usa todas las filas.
        order = np.argsort(x)
        fig, ax = plt.subplots(figsize=(9, 5.5))
        ax.scatter(x, y, label="Observaciones", color="#1f77b4", alpha=0.85)
        ax.plot(np.asarray(x)[order], result.predicted[order], label=f"Ajuste polinómico (
            grado {self.degree})", color="#d62728")
        ax.set(title="Regresión polinómica", xlabel="x", ylabel="y")
        ax.grid(True, alpha=0.25)
        ax.legend()
        fig.tight_layout()
        fig.savefig(path, dpi=160)
        plt.close(fig)


class PolynomialSearchEstimator(RegressorMixin, BaseEstimator):
    """Adaptador para que la clase NumPy se pueda usar en Pipeline y CV.

    scikit-learn llama a fit(X, y) en cada pliegue y a predict(X) para puntuarlo.
    Este adaptador convierte la columna X al formato que espera PolynomialRegressor.
    """

    def __init__(self, degree=2, rcond=None, full=False, w=None, cov=False):
        self.degree = degree
        self.rcond = rcond
        self.full = full
        self.w = w
        self.cov = cov

    def fit(self, X, y, sample_weight=None):
        # Un polinomio de esta práctica tiene un único predictor, x. sklearn lo
        # entrega como matriz (filas, columnas); la clase base usa un vector.
        x = np.asarray(X, dtype=float)
        if x.ndim == 2 and x.shape[1] == 1:
            x = x[:, 0]
        if x.ndim != 1:
            raise ValueError("PolynomialSearchEstimator acepta exactamente una columna X.")
        # sample_weight se recorta automáticamente por RandomizedSearchCV para
        # cada pliegue. self.w queda como opción para un ajuste sin validación.
        weights = sample_weight if sample_weight is not None else self.w
        self.regressor_ = PolynomialRegressor(x, x, y, y, degree=self.degree)
        self.fit_result_ = self.regressor_.fit(
            rcond=self.rcond, full=self.full, w=weights, cov=self.cov
        )
        self.coef_ = self.regressor_.coefficients_
        self.n_features_in_ = 1
        return self

    def predict(self, X):
```

```
        if not hasattr(self, "regressor_"):
            raise ValueError("El estimador aún no está entrenado.")
        x = np.asarray(X, dtype=float)
        if x.ndim == 2 and x.shape[1] == 1:
            x = x[:, 0]
        if x.ndim != 1:
            raise ValueError("PolynomialSearchEstimator acepta exactamente una columna X.")
        return self.regressor_.predict(x)


# RandomizedSearchCV elige valores de estas listas para el paso 'polynomial'.
# Los nombres llevan 'polynomial__' porque ése es el nombre del paso del Pipeline.
# 'w' se deja en None porque una matriz de pesos de tamaño fijo no es válida al
# cambiar los pliegues CV; pase los pesos reales a search.fit como sample_weight.
POLYNOMIAL_PARAM_DISTRIBUTIONS = {
    # Cada lista da las opciones que RandomizedSearchCV puede probar.
    "polynomial__degree": [1, 2, 3, 4, 5],
    # rcond ajusta qué valores singulares pequeños se descartan.
    "polynomial__rcond": [None, 1e-12, 1e-10, 1e-8],
    # full y cov solicitan información diagnóstica del ajuste.
    "polynomial__full": [False, True],
    # Una lista w fija no sirve en pliegues de tamaños distintos; use sample_weight.
    "polynomial__w": [None],
    "polynomial__cov": [False, "unscaled"],
}


def build_polynomial_search(n_iter=20, cv=5, random_state=42, param_distributions=None):
    """Construye la búsqueda reproducible para una variable x."""
    # Pipeline reúne el estimador bajo un nombre estable para que sus parámetros
    # puedan escribirse en el diccionario de búsqueda con el prefijo 'polynomial__'.
    pipeline = Pipeline([("polynomial", PolynomialSearchEstimator())])
    return RandomizedSearchCV(
        estimator=pipeline,
        param_distributions=(param_distributions if param_distributions is not None
                             else POLYNOMIAL_PARAM_DISTRIBUTIONS),
        n_iter=n_iter,
        cv=cv,
        random_state=random_state,
        scoring="neg_mean_squared_error",
        refit=True,
    )
```

D. Clase de Random Forest

La clase encapsula el preprocesamiento, entrenamiento, evaluación y generación de artefactos.

Listing 4: Código de regression/random_forest_regressor.py

```
"""Entrenamiento y evaluación de un Random Forest para regresión tabular."""
from __future__ import annotations

import pickle
from dataclasses import dataclass
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

```
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder


@dataclass
class RandomForestResult:
    model: Pipeline
    metrics: dict
    predictions: pd.DataFrame
    feature_importances: list[dict]
    feature_columns: list[str]
    target: str


class RandomForestRegressorModel:
    """Preprocesa variables mixtas, entrena el bosque y produce métricas."""

    def __init__(self, n_estimators=200, max_depth=None, test_size=0.2, random_state=42):
        if isinstance(n_estimators, bool) or not isinstance(n_estimators, int) or not 10
            <= n_estimators <= 1000:
            raise ValueError("n_estimators debe ser un entero entre 10 y 1000.")
        if max_depth is not None and (isinstance(max_depth, bool) or not isinstance(
            max_depth, int) or not 1 <= max_depth <= 100):
            raise ValueError("max_depth debe ser null o un entero entre 1 y 100.")
        if isinstance(test_size, bool) or not isinstance(test_size, (int, float)) or not
            0.1 <= test_size <= 0.4:
            raise ValueError("test_size debe estar entre 0.1 y 0.4.")
        if isinstance(random_state, bool) or not isinstance(random_state, int) or
            random_state < 0:
            raise ValueError("random_state debe ser un entero no negativo.")
        self.n_estimators = n_estimators
        self.max_depth = max_depth
        self.test_size = float(test_size)
        self.random_state = random_state

    def fit(self, frame: pd.DataFrame, target: str = "y") -> RandomForestResult:
        # Primero se separa la columna que queremos predecir del resto de la tabla.
        if target not in frame.columns:
            raise ValueError(f"No existe la columna objetivo '{target}'.")
        # El ID solo enlaza resultados con expedientes; excluirlo evita memorizar claves.
        id_column = "expediente_id" if "expediente_id" in frame.columns else None
        feature_columns = [col for col in frame.columns if col not in {target, id_column}]
        if not feature_columns:
            raise ValueError("El CSV debe aportar al menos una columna predictora además
                del objetivo.")
        data = frame.copy()
        data[target] = pd.to_numeric(data[target], errors="coerce")
        valid_target = data[target].notna() & np.isfinite(data[target].to_numpy(dtype=
            float))
        data = data.loc[valid_target].reset_index(drop=True)
        if len(data) < 10:
            raise ValueError("Se requieren al menos 10 filas con objetivo numérico para
                entrenar y evaluar.")
```

```
X = data[feature_columns].copy()
y = data[target].astype(float)
# El texto que solo contiene números se convierte a número; el resto se
# tratará como categoría, por ejemplo el tipo de procedimiento.
for column in feature_columns:
    if X[column].dtype == object:
        converted = pd.to_numeric(X[column], errors="coerce")
        non_empty = X[column].notna() & (X[column].astype(str).str.strip() != "")
        if non_empty.any() and converted[non_empty].notna().all():
            X[column] = converted
numeric_columns = X.select_dtypes(include=["number", "bool"]).columns.tolist()
categorical_columns = [col for col in feature_columns if col not in
    numeric_columns]
# Cada grupo de columnas necesita su propio tratamiento de valores ausentes.
transformers = []
if numeric_columns:
    transformers.append(("numeric", Pipeline([("imputer", SimpleImputer(strategy="
        median"))]), numeric_columns))
if categorical_columns:
    transformers.append(("categorical", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]), categorical_columns))
# Pipeline repite automáticamente el mismo preprocesamiento al entrenar y
# predecir, antes de ejecutar RandomForestRegressor.
pipeline = Pipeline([
    ("preprocess", ColumnTransformer(transformers=transformers, remainder="drop")),
    ("random_forest", RandomForestRegressor(
        n_estimators=self.n_estimators, max_depth=self.max_depth,
        random_state=self.random_state, n_jobs=-1,
    )),
])
# Reservamos filas de prueba que no participan en el entrenamiento; las
# métricas posteriores estiman cómo funciona el modelo con datos nuevos.
indices = np.arange(len(y))
train_idx, test_idx = train_test_split(indices, test_size=self.test_size,
    random_state=self.random_state)
pipeline.fit(X.iloc[train_idx], y.iloc[train_idx])
predicted = pipeline.predict(X.iloc[test_idx])
observed = y.iloc[test_idx].to_numpy(dtype=float)
residuals = observed - predicted
metrics = {
    "r2": float(r2_score(observed, predicted)),
    "rmse": float(np.sqrt(mean_squared_error(observed, predicted))),
    "mae": float(mean_absolute_error(observed, predicted)),
}
predictions = pd.DataFrame({"y_observado": observed, "y_predicho": predicted, "
    residuo": residuals})
if id_column:
    predictions.insert(0, id_column, data[id_column].iloc[test_idx].to_numpy())
# One-hot encoding puede expandir una categoría a varias columnas; usamos
# los nombres transformados para etiquetar la importancia del bosque.
names = pipeline.named_steps["preprocess"].get_feature_names_out()
scores = pipeline.named_steps["random_forest"].feature_importances_
importance = [{"feature": str(name), "importance": float(score)} for name, score
    in
              sorted(zip(names, scores), key=lambda item: item[1], reverse=True)]
return RandomForestResult(pipeline, metrics, predictions, importance,
    feature_columns, target)
```

```
    @staticmethod
    def save(path: Path, result: RandomForestResult):
        # Cargar pickle ejecuta código: aceptar únicamente ficheros de origen confiable.
        with Path(path).open("wb") as stream:
            pickle.dump(result.model, stream, protocol=pickle.HIGHEST_PROTOCOL)

    @staticmethod
    def write_predictions(path: Path, result: RandomForestResult):
        result.predictions.to_csv(path, index=False, encoding="utf-8")

    @staticmethod
    def write_plot(path: Path, result: RandomForestResult):
        fig, axes = plt.subplots(1, 2, figsize=(12, 5))
        observed = result.predictions["y_observado"].to_numpy()
        predicted = result.predictions["y_predicho"].to_numpy()
        limits = [min(observed.min(), predicted.min()), max(observed.max(), predicted.max
            ())]
        axes[0].scatter(observed, predicted, alpha=0.7, color="#1f77b4")
        axes[0].plot(limits, limits, "--", color="#d62728", label="Predicción ideal")
        axes[0].set(title="Observado frente a predicho", xlabel="Valor observado", ylabel=
            "Valor predicho")
        axes[0].legend()
        axes[0].grid(True, alpha=0.25)
        top = result.feature_importances[:15][::-1]
        axes[1].barh([row["feature"] for row in top], [row["importance"] for row in top],
            color="#2ca02c")
        axes[1].set(title="Importancia de variables (15 principales)", xlabel="Importancia
            ")
        axes[1].grid(axis="x", alpha=0.25)
        fig.tight_layout()
        fig.savefig(path, dpi=160)
        plt.close(fig)
```